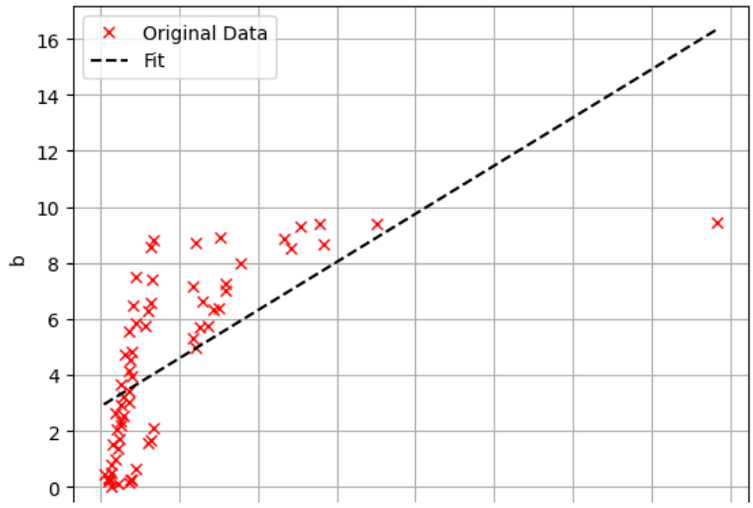
```
##hw1
##Sherry Yang


import csv
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd



data = pd.read_csv('hw1_data.csv', header=None)
t = data.iloc[0:65,0]
y = data.iloc[0:65,1]


if __name__ == '__main__':
    print(0)
```

```
0
```

New Section

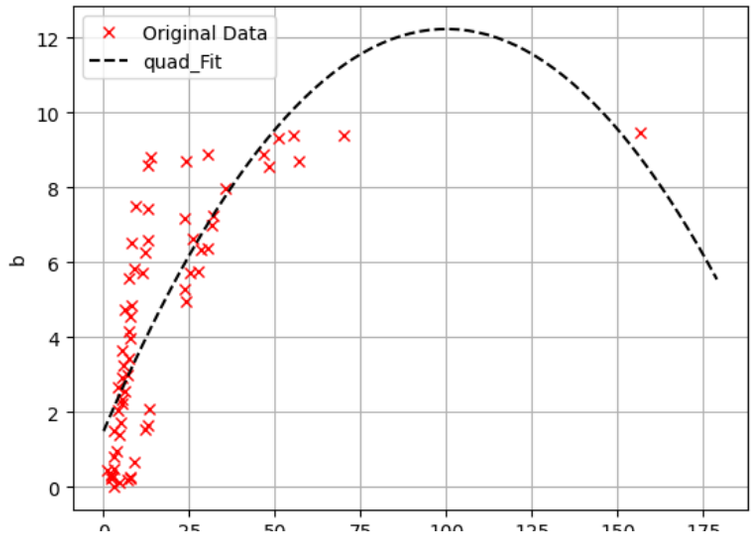
```
def calculate_r_squared(true, pred):
    mean_y = sum(true) / len(true)

    ssr = sum((true - pred) ** 2 for true, pred in zip(true,pred))
    sst = sum((true - mean_y) ** 2 for true in true)


    r_squared = 1 - (ssr / sst)

    return r_squared


# def calculate_r_squared(true, pred):
#     TSS = sum((y[i] -np.mean(y)**2)( )
#     RSS = sum((pred - np.mean(y[i])**2)
#     R2 = 1 - RSS/TSS

#     return R2


##linear model
A = np.column_stack((np.ones_like(t),t))

##calculate coef
coef = np.linalg.inv(A.T @ A) @ A.T @ y
print("coef", coef)
predicted = coef[0] + coef[1]*t


last_point_linear = coef[0] + coef[1]*data.iloc[65,0]
print("last_point_linear", last_point_linear)

r_squared_lin=calculate_r_squared(y,predicted)
print("r^2", r_squared_lin)

## R squared value = 0.42, fit is pretty bad

# Plot the original data
plt.plot(t, y, 'rx', label='Original Data')

# Plot the linear fit
plt.plot(t, predicted, 'k--', label='Fit')

plt.xlabel('t')
plt.ylabel('b')
plt.legend()
plt.grid(True)
plt.show()
```

```
coef [2.8629439  0.08611653]
last_point_linear 17.93664007654747
r^2 0.42084081227244885
```



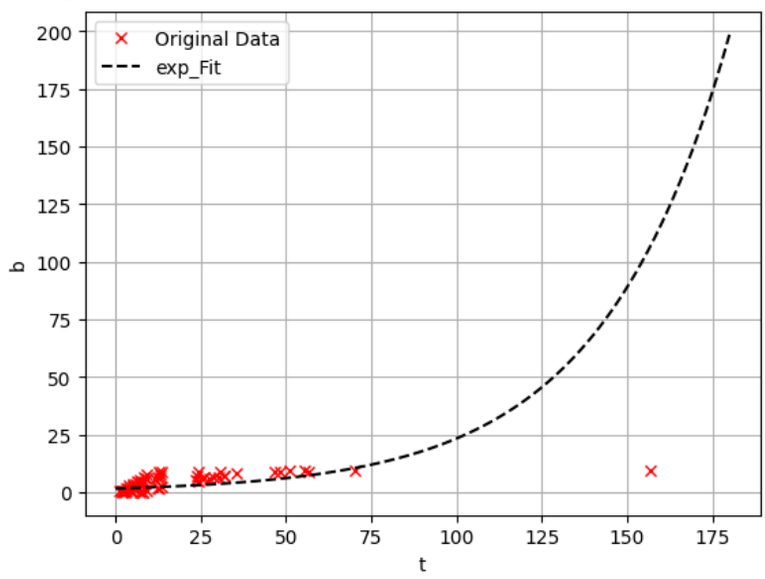

```
##quadratic model

A2 = np.column_stack((np.ones_like(t),t,t**2))
coef = np.linalg.inv(A2.T @ A2) @ A2.T @ y
print("coef",coef)


interval = np.linspace(0,179,65)


def new_data(t):
  predicted_quad = coef[0] + coef[1]*t + coef[2]*t**2
  return(predicted_quad)

last_point_quadratic = coef[0] + coef[1]*data.iloc[65,0] + coef[2]*(data.iloc[65,0])**2
print("last_point_quadratic", last_point_quadratic)

r_squared_quad = calculate_r_squared(y,new_data(interval))
print("r^2 quad", r_squared_quad)
## R squared value = 0.64, fit is better than linear, but still not great

# Plot the original data
plt.plot(t, y, 'rx', label='Original Data')

# Plot the linear fit
plt.plot(interval, new_data(interval), 'k--', label='quad_Fit')

plt.xlabel('t')
plt.ylabel('b')
plt.legend()
plt.grid(True)
plt.show()
```

```
coef [ 1.49345447e+00  2.14578227e-01 -1.07228826e-03]
last_point_quadratic 6.199649956908175
r^2 quad -2.6815231891392983
```



```
##exponential model


z = np.log(y)

A3 = np.column_stack((np.ones_like(t), t))
coef = np.linalg.inv(A3.T @ A3) @ A3.T @ z
print("coef",coef)


interval = np.linspace(0,179,65)


def equation(t):
  predicted=coef[0] + coef[1] * t
  return(predicted)

print("last_point_exp", np.exp(equation(data.iloc[65,0])))

r_squared_exp = calculate_r_squared(z,equation(interval))
print("r^2 exp", np.exp(r_squared_exp))
## R squared value = 0.21, most horrible out of all three. even though only the farthest point is awfully off, that throws of

# Plot the original data
plt.plot(t, y, 'rx', label='Original Data')


t_interval = np.linspace(0,180,100)
plt.plot(t_interval, np.exp(equation(t_interval)), 'k--', label='exp_Fit')

plt.xlabel('t')
plt.ylabel('b')
plt.legend()
plt.grid(True)
plt.show()
```

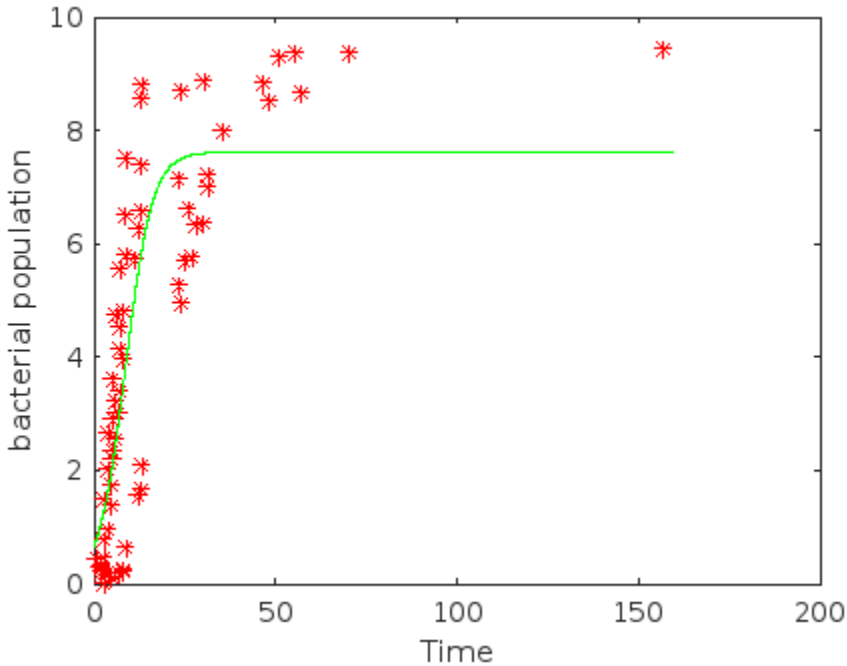
```
coef [0.47307808 0.02676555]
last_point_exp 173.82957753532688
r^2 exp 0.20588626540355576
```



New Section

```
%Gauss Newton Example  problem 2
close all
clear all

data=csvread('hw1_data.csv');
t = data(1:65,1); %time
y = data(1:65,2); % population
tol = 10^-6;
err = 1;
x = [1; 10; 0.1]; %initial guess, N,k,r
%residuals = y - g = y - N*k*exp(r*t)
% J = e ^ (x2t) tx1e^(x2 t) Jacobian
while err > tol;
    g = (x(1).*x(2).*exp(x(3).*t))./(x(2) + x(1).*(exp(x(3).*t)-1));
    res = y - g;
    dJ_dN = x(2).^2.*exp(x(3).*t)./(((exp(x(3).*t)-1).*x(1)+x(2)).^2);
    dJ_dk = x(1).^2.*(exp(x(3).*t)-1).*exp(x(3).*t)./
((x(2)+x(1).*exp(x(3).*t)-1)).^2;
    dJ_dr = (x(1).*x(2).*(x(2)-x(1)).*t.*exp(x(3).*t))./((x(1).*exp(x(3).*t)
+x(2)-x(1)).^2);
    J = [dJ_dN dJ_dk dJ_dr]; %Jacobian
    p = inv(J'*J)*J'*res; %invert normal equations to solve for update
    x = x + p; %update x
    err = norm(p,2); %
end

fprintf('N,K,R: %.4f/', x)


RSS = sum(res.^2);
TSS = sum((y-mean(y)).^2);
R_squared = 1-(RSS/TSS);
%R sqaured = 0.676, similar performance to the quadratic model, not much
improvement

figure()
plot(t,y,'r*')
hold on
T = 0:0.01:160;
yrg = (x(1).*x(2).*exp(x(3).*T))./(x(2) + x(1).*(exp(x(3).*T)-1));
plot(T, yrg,'g-')
xlabel('Time')
ylabel('bacterial population')
hold off
```

N,K,R: 0.6693/N,K,R: 7.6180/N,K,R: 0.2700/



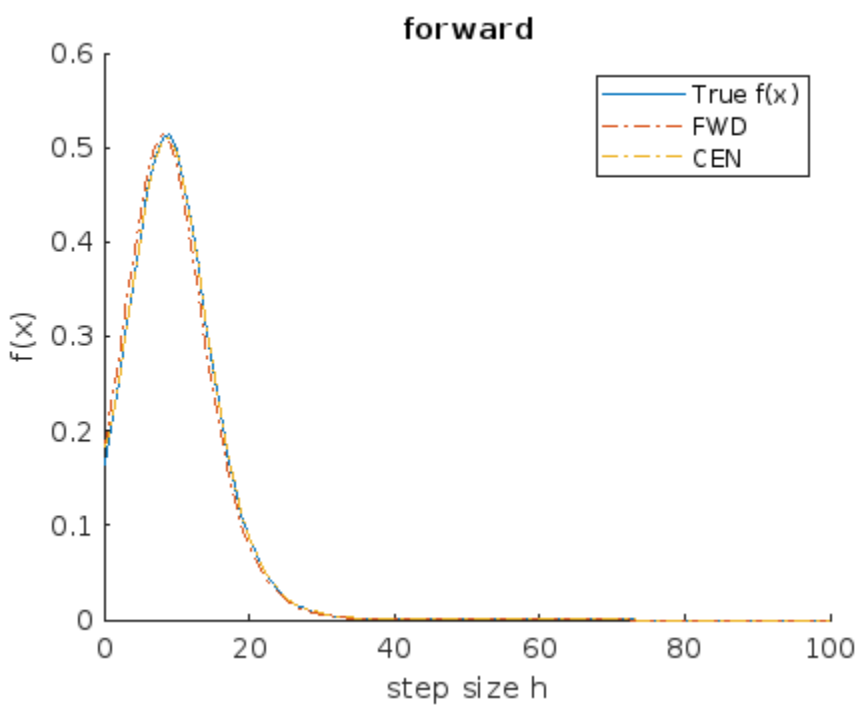
Problem 1 Q3

```
t = linspace(0,100,101);
g = (x(1).*x(2).*exp(x(3).*t))./(x(2) + x(1).*(exp(x(3).*t)-1));


fwd = zeros(size(t));
ctd = zeros(size(t));

for i = 1:100
    fwd(i) = (g(i + 1) - g(i)) / (t(i + 1) - t(i));
    fwd(101) = (g(101)-g(100))/(t(101)-t(100));
end



%central

for i = 2:100
    ctd(i) = (g(i + 1) - g(i - 1)) / ((t(i + 1) - t(i-1)));
end
ctd(1) = (g(2)-g(1)) / (t(2)-t(1));
ctd(101) = g(101)-g(100)/ ( (t(101)-t(100)));

ftrue = x(3)*g .* (1-g/ x(2));

figure()
hold on
plot(t, ftrue)
plot(t, fwd, '-.')
```

```
plot (t,ctd, '-.')

xlabel('step size h')
ylabel('f(x)')
title('forward')
legend('True f(x)','FWD','CEN')
hold off



%problem 2

data1=readtable('hw1_ex2.csv');
rec = data1.recA;
lex = data1.lexA;
sul = data1.sulA;
umu = data1.umuDC;
dinB = data1.dinB;

A4= [rec, lex, sul, umu];

coef = inv(A4'*A4)*A4'*dinB;

fprintf('coefficient: %.4f/', coef)



predicted = A4*coef;
res = dinB - predicted;

RSS = sum(res.^2);
TSS = sum((dinB-mean(dinB)).^2);
R_squared = 1-(RSS/TSS);


% function c = concentration(rec1, lex1, sul1, umu1, coef)
%     predicted_dinB_conc = coef(1) * rec1 + coef(2) * lex1 + coef(3) * sul1
+ coef(4) * umu1;
%
%     c = predicted_dinB_conc;
% end

c1 = [6.2, 7, 7.4, 7];
c2 = [3.1, 7, 10.8, 7];
c3 = [5.4, 10.1, 8.3, 12];
c4 = [15, 15, 15, 15];

C1 = c1*coef;
C2 = c2*coef;
C3 = c3*coef;
C4 = c4*coef;

c5 = [100, 100, 100, 100];
C5 = c5*coef;
```

```
fprintf('condition 1: %.4f/', C1)
fprintf('condition 2: %.4f/', C2)
fprintf('condition 3: %.4f/', C3)
fprintf('condition 4: %.4f/', C4)


% dinB = 6.951 e3

% part 2 question 3
fprintf('condition 5: %.4f/', C5)
% its not a good approximation because the value 100 100 100 100 is way out of
% the dataset that the model was built on. the model can literally go anyway
% after the max number of x value. it's out of the domain of the independent
% variable. To improve this model, we need to collect dataset where the
% concentration of the regulatory element is around 100 to capture a better
% model.
```

coefficient: 169.7761/coefficient: -93.0891/coefficient: -47.2619/ coefficient: 40.0855/condition 1: 331.8494/condition 2: -355.1471/condition 3: 65.3443/condition 4: 1042.6608/condition 5: 6951.0723/



Published with MATLAB® R2023b